Lab 09 Team Management (Phase 0 of Assignment)

Q1 : Start-up version (`Team and Employee, Company, Day, SystemDate`)
Q2-3 : Command handling (Hire Employee, Create Team, Listing, Change System Date, Undo, Redo)

Q1. We will model a Team Management Program (startup version) in this exercise.

Classes to be created:

Don't hurry !
Code will be given in P.2

- class **Employee**, **Team**
- class **Company** //As singleton

```
public class Company
{
  private ArrayList<Employee> allEmployees;
  private ArrayList<Team> allTeams;
  private static Company instance = new Company(); //The instance created when the class is loaded.
  ..
}
```

- public class **Day**
- **class SystemDate extends Day //As singleton**

```
public class SystemDate extends Day
{
  private static SystemDate instance;
  public static void createTheInstance(String sDay)
  {
    ..
    instance = new SystemDate(..);        //The instance is created when this method is called like:
  }                                       SystemDate.createTheInstance("01-Jan-2025");
  ..
}
```

The program (`main()`):

```
public static void main(String[] args)
{
[Line 1]   SystemDate.createTheInstance("01-Jan-2025");

[Line 2]   Company co = Company.getInstance();
[Line 3]   co.createEmployee("Bob", 30);
[Line 4]   co.createEmployee("John", 30);
[Line 5]   co.createEmployee("Grace", 30);

[Line 6]   co.createTeam("Customer Relationship Team", "Bob");
[Line 7]   co.createTeam("Strategic Planning Team", "John");

[Line 8]   SystemDate.getInstance().set("01-Feb-2025"); // set system date to 1/2/2025
[Line 9]   co.createTeam("Clerical Support Team", "Grace");

[Line 10]  co.listTeams();
}
```

Output

```
Team Name                    Leader    Setup Date
Clerical Support Team        Grace     1-Feb-2025
Customer Relationship Team   Bob       1-Jan-2025
Strategic Planning Team      John      1-Jan-2025
```

Explanation:

The program

- Implements a team system used by a Company (only one Company object, therefore singleton):
  - the Company instance is created when the Company class is loaded.
- Maintains a system date object (only one SystemDate object, therefore singleton):
  - inherit the Day class
  - the instance is created when main() runs: SystemDate.createTheInstance("01-Jan-2025"); [Line 1]
- Adds 3 employees (with entitled annual leaves) and 2 teams (with team heads) on 01-Jan-2025 [Line 3-7]
  In this simplified version, all teams are one-man-teams. We will add team members in the assignment.
- System date advances to 01-Feb-2025 [Line 8], then one team added [Line 9]
- List all teams [Line 10]

Step 1. Revise the previous **Day class** - `cloneable` and conversion to/from `dd-MMM-yyyy`

Like `01-Mar-2025`

Your tasks: Download the given `Day.java` file. Follow the steps below.

Keep original fields:

```
private int year;
private int month;
private int day;
```

Add new constant field (*static*, *final*):

Will be used in string conversion

```
private static final String MonthNames="JanFebMarAprMayJunJulAugSepOctNovDec";
```

Add / revise methods:

(1) Add a `set` method and a `constructor` for string conversion:

```
public void set(String sDay) { //Set year,month,day based on a string like 01-Mar-2025
   String[] sDayParts = sDay.split("-");
   this.day = ..; //Apply Integer.parseInt for sDayParts[0];
   this.year = ..;
   this.month = MonthNames.indexOf(sDayParts[1])/3+1;
}
```

```
public Day(String sDay) {..} //Constructor, simply call set(sDay)
```

(2) Change the `toString` method to make use of `MonthNames`

```
@Override
public String toString() {
   return day+"-"+ MonthNames.substring(____,____) + "-"+ year; // (month-1)*3,(month)*3
}
```

(3) Make Day `cloneable` and add the `clone` method

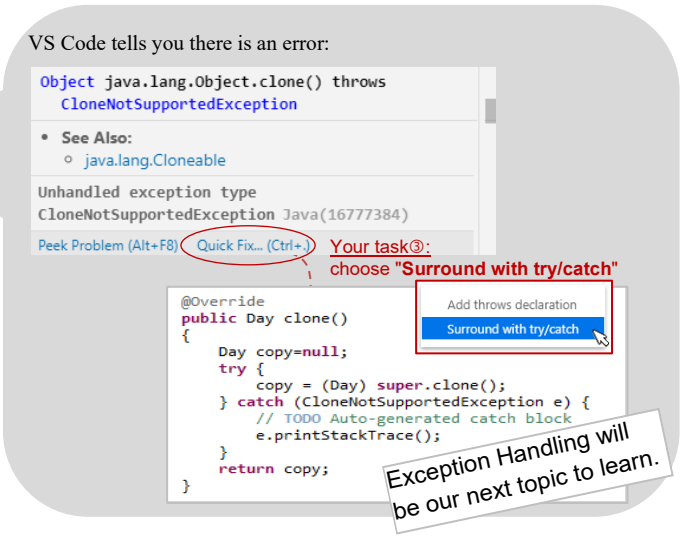
①
```
public class Day implements Cloneable
```

②
```
@Override
public Day clone() {

   Day copy=null;
   copy = (Day) super.clone();

   return copy;
}
```

VS Code tells you there is an error:



[Note]
Please learn from steps (1)-(3) when you apply them.
Do not remove other original methods.

Step 2. Create the **`SystemDate` class** - inherits `Day`, singleton*, instance created on demand#

Inherits the Day class : `public class SystemDate extends Day`

Static field for the Singleton* : `private static SystemDate instance;`

Private constructor for Singleton* : `private SystemDate(String sDay) {super(sDay);}`

Simply call the constructor of the Day superclass, which sets year,month,day. SystemDate itself doesn't add any instance field (ie. nonstatic fields), therefore no need to do further initialization of instance field.

Static method for getting Singleton* : `public static SystemDate getInstance()` {return _____;}

Public static method for creating the Singleton # :

```
public static void createTheInstance(String sDay) {
   if (instance==null) //make sure only one instance can be created (Singleton)
       instance = new SystemDate(sDay);
   else
       System.out.println("Cannot create one more system date instance.");
}
```

[Note] Please learn from the above when you apply them.

Step 3. Create the **`Employee` class**

Instance fields:

```
private String name;
private int annualLeaves; // Entitled annual leaves, given as a count of days
```

Constructor:

```
public Employee(String n, int yle) {
    .. //Set name, annualLeaves
}
```

Other methods:

```
public String getName() { /*simply return the name string */}

public static Employee searchEmployee(ArrayList<Employee> list,
    String nameToSearch) {
    /*search the arrayList for the employee with the given name */
}
```

```
.. more will be added in later steps.
```

[Note] Please learn from the above when you apply them.

Step 4. Create the **`Team` class**

Instance fields:

```
private String teamName;
private Employee head;
private Day dateSetup; //the date this team was setup
```

Constructor:

```
public Team(String n, Employee hd)
{
    ..//Set teamName, head, setup date
}
```

dateSetup should get a copy of System Date:
SystemDate.getInstance().clone();

`list()` - list all teams:

```
public static void list(ArrayList<Team> list)
{
    //Learn: "-" means left-aligned
    System.out.printf("%-30s%-10s%-13s\n", "Team Name", "Leader", "Setup Date");
    for (Team t : list)
        System.out.printf(..); //display t.teamName, etc..
}
```

Type the hyphens ("-") on your own. The ones in .pdf do not work.

Output

```
Team Name                      Leader    Setup Date
Clerical Support Team          Grace     1-Feb-2025
Customer Relationship Team     Bob       1-Jan-2025
Strategic Planning Team        John      1-Jan-2025
```

```
.. more will be added in later steps.
```

[Note] Please learn from the above when you apply them.

Step 5. Finish the **`Company` class** - Singleton*, holds arrays of employees and teams;

Java API
```
import java.util.ArrayList;
import java.util.Collections; //Provides sorting
```

Instance field:
```
private ArrayList<Employee> allEmployees;
private ArrayList<Team> allTeams;
```
- Instances will be created in the constructor

- Instance created when the class is loaded.

Static field for the Singleton* : `private static Company instance = new Company();`

Private constructor for Singleton* : `private Company() { /* create the 2 arrayLists */ }`

Static method for getting Singleton* : `public static Company getInstance() { return ___; }`

Add other methods:

```
public void listTeams()  //See how it is called in main()
{
    Team.list(__________); // allTeams
}
```

```
public Employee createEmployee(_____________) //See how it is called in main()
{
    Employee e = new Employee(____________);
    allEmployees.add(____);
    Collections.sort(_________);  // allEmployees
    return e;            //  the return value is useful for later undoable command.
}
```
Currently there is an error concerning the argument. To fix, we will make Employee comparable (Step 6)

```
public Team createTeam(_____________) //See how it is called in main()
{
    Employee e = Employee.searchEmployee(___________________);
    Team t = new Team(____________);
    allTeams.add(_____);
    Collections.sort(_________); // allTeams
    return t;            //  the return value is useful for later undoable command.
}
```
Currently there is an error concerning the argument. To fix, we will make Team comparable (Step 7)

[Note] Please learn from the above when you apply them.

Step 6. Revise the **`Employee`** class - Make Employee objects sortable

Implement the `Comparable` interface and add the `compareTo` method

-
```
public class Employee implements Comparable<Employee>
```

-
```
public int compareTo(Employee another)
{
   if (this.name.equals(another.name))  return 0;
   else if (this.name.compareTo(another.name)>0) return 1;
   else return -1;
}
```
add @Override

or simply:

-
```
public int compareTo(Employee another)
{
  return this.name.compareTo(another.name);
}
```
add @Override

[Note] Please learn from the above when you apply them.

Step 7. Revise the **`Team`** class - Make Team objects sortable by team names (See Step 6)

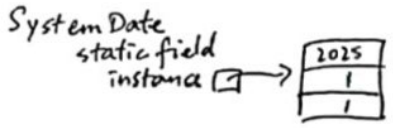
Step 8. Test your program. It should show:

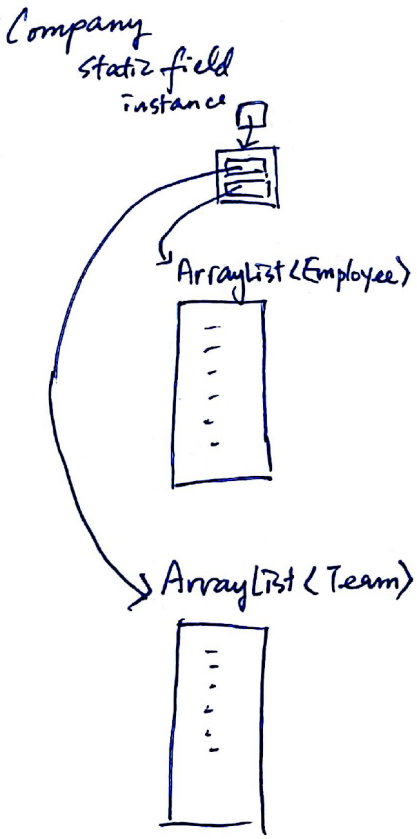
Output

```
Team Name                    Leader    Setup Date
Clerical Support Team        Grace     1-Feb-2025
Customer Relationship Team   Bob       1-Jan-2025
Strategic Planning Team      John      1-Jan-2025
```

Lab09 - Progress

[Drawing] Draw all fields and objects that appear when the following program runs:






```
public static void main(String [] args)
{
  SystemDate.createTheInstance("01-Jan-2025");

  Company co = Company.getInstance();
  co.createEmployee("Bob");
  co.createEmployee("John");

  co.createTeam("Sales", "Bob");

  SystemDate.getInstance().set("01-Feb-2025");
  co.createTeam("Production", "John");

  co.listTeams();
}
```

```
public class SystemDate extends Day ← int year, month, day
{
  private static SystemDate instance;

  public static void createTheInstance(String sDay)
  {
    if (instance==null)
      instance = new SystemDate(sDay);
    else
      System.out.println("Cannot create one more ..");
  }
  ...
}
```

```
public class Employee ..
{
  ..//name

  ..
}
```

```
public class Team ..
{
  ..//teamName, head, dateSetup

  ..
}
```

```
public class Company
{
  private ArrayList<Employee> allEmployees;
  private ArrayList<Team> allTeams;

  private static Company instance = new Company();

  private Company()
  {
    allEmployees = new..; allTeams = new..;
  }

  public .. createEmployee(..)
  {
    ..
    allEmployees.add(..);
    Collections.sort(allEmployees);
    ..
  }
  ...
}
```

Q2 Command handling (Hire employee, listing, undo, redo)

- Please review / redo Lab08 all Q2-Q6 before doing this question.

Refer to given Main,java, testcase (data1.txt) and rundown at the course web, extend your program for Q1 to handle the commands as follows:

Sample input file:

First line is initial system date

Then the commands
- Hire employees (name, entitled annual leaves)
- listing of employees
- undo
- redo

Blank lines added for easy reading.

data1.txt

```
startNewDay|01-Jan-2025

hire|Bob|30
hire|John|30
hire|Grace|30

listEmployees

undo
undo
undo
undo

listEmployees

redo
redo
listEmployees

hire|Helena|40
listEmployees
redo
```

Note:

For undo/redo of the hire command,
add the following methods in *Company.java*:

```
public void addEmployee(Employee e) {
      allEmployees.add(_);
      Collections.sort(_____);
}

public void removeEmployee(Employee e) {
      allEmployees.remove(_); //.remove is a method of ArrayList
}
```

Sample rundown:

```
Please input the file pathname: data1.txt

> startNewDay|01-Jan-2025

> hire|Bob|30
Done.

> hire|John|30
Done.

> hire|Grace|30
Done.

> listEmployees
Bob (Entitled Annual Leaves: 30 days)
Grace (Entitled Annual Leaves: 30 days)
John (Entitled Annual Leaves: 30 days)

> undo

> undo

> undo

> undo
Nothing to undo.

> listEmployees

> redo

> redo

> listEmployees
Bob (Entitled Annual Leaves: 30 days)
John (Entitled Annual Leaves: 30 days)

> hire|Helena|40
Done.

> listEmployees
Bob (Entitled Annual Leaves: 30 days)
Helena (Entitled Annual Leaves: 40 days)
John (Entitled Annual Leaves: 30 days)

> redo
Nothing to redo.
```

Q3 Command handling (Setup team, listing, change system date, undo, redo)

Refer to given (data2.txt) and rundown at the course web, extend your program for Q2 to handle the commands as follows:

Sample input file:

New commands:
- startNewDay
- setupTeam (team name, head)
- listTeams

Blank lines added for easy reading.

data2.txt

```
startNewDay|01-Jan-2025

hire|Bob|30
hire|John|30
hire|Grace|30

setupTeam|Customer Relationship Team|Bob
setupTeam|Strategic Planning Team|John

startNewDay|01-Feb-2025
setupTeam|Clerical Support Team|Grace

listTeams

undo
undo
undo
undo
undo
undo
undo
undo

listTeams

redo
redo
redo
redo
redo
listTeams

setupTeam|Strategic Planning Team|Grace
listTeams
redo
```

Sample rundown:

```
Please input the file pathname: data2.txt

> startNewDay|01-Jan-2025

> hire|Bob|30
Done.

> hire|John|30
Done.

> hire|Grace|30
Done.

> setupTeam|Customer Relationship Team|Bob
Done.

> setupTeam|Strategic Planning Team|John
Done.

> startNewDay|01-Feb-2025
Done.

> setupTeam|Clerical Support Team|Grace
Done.

> listTeams
Team Name                      Leader    Setup Date
Clerical Support Team          Grace     1-Feb-2025
Customer Relationship Team     Bob       1-Jan-2025
Strategic Planning Team        John      1-Jan-2025

> undo
> undo
> undo
> undo
> undo
> undo
> undo
> undo
Nothing to undo.

> listTeams
Team Name                      Leader    Setup Date

> redo
> redo
> redo
> redo
> redo

> listTeams
Team Name                      Leader    Setup Date
Customer Relationship Team     Bob       1-Jan-2025
Strategic Planning Team        John      1-Jan-2025

> setupTeam|Strategic Planning Team|Grace
Done.

> listTeams
Team Name                      Leader    Setup Date
Customer Relationship Team     Bob       1-Jan-2025
Strategic Planning Team        John      1-Jan-2025
Strategic Planning Team        Grace     1-Jan-2025

> redo
Nothing to redo.
```